

Predicting Air Pressure System (APS) Failures in Scania Trucks

Alena Fisher

CSCI 460 – Introduction to Applied Machine Learning

Dr. Edgar Eduardo Ceh Varela

May 3, 2026

Abstract

This project explores the use of machine learning processes to predict failures in the Air Pressure System (APS) of heavy Scania trucks. Scania trucks rely on the APS to operate the brake and gear shifting systems. Accurate prediction and early detection of APS failures are essential, as undetected failures can lead to costly repairs, while inaccurate predictions will result in unnecessary and expensive maintenance (Taghandiki et al., 2023).

The dataset used in this project is from sensors on the trucks, collected during normal usage. It contains 60,000 training instances, 59,000 of which belong to the negative class, and 1,000 belong to the positive class, along with a dataset of 16,000 test instances. There are 171 attributes that represent various system measurements for each instance. A key challenge in this dataset is the severe class imbalance, where only a small fraction of instances represent APS failures.

An Exploratory Data Analysis (EDA) was conducted to understand the structure and characteristics of the dataset, including feature distribution, data points for model building, and missing values. Several preprocessing techniques were used to prepare the dataset, including handling missing features, removing attributes with excessive missing data, and standardizing features. Six machine learning models were chosen to analyze the dataset: Logistic Regression, Decision Tree, K-Nearest Neighbor (KNN), Support Vector Machine (SVM), Random Forest, and XGBoost.

The models' performances were evaluated using the metrics accuracy, precision, recall, and F1-score, with particular emphasis on F1-score due to the class imbalance. Among all models, XGBoost had the best overall performance with an F1-score of 0.8595, making it the most effective model for APS failure detection. The results show the importance of handling class imbalance and the effectiveness of ensemble algorithms in predictive maintenance applications.

Keywords: Air Pressure System (APS), predictive maintenance, binary classification, machine learning

1. Introduction

1.1 Problem Definition

Traditional maintenance approaches are often done by humans, which can be time-consuming and flawed. It is easy to miss something or miscommunicate. This is why there has been a rise in predictive maintenance using machine learning; by collecting and analyzing data collected from sensors, machine learning models are able to identify patterns in the data and predict equipment malfunctions or failure before they occur. Early and accurate detection allows for the issues to be addressed in a timely manner, which reduces downtime, repair costs, and can prevent a complete system failure (Hussain S.A et al., 2024).

The Air Pressure System (APS) is responsible for generating pressurized air that the Scania Trucks use for the breaking and gear shifting systems. A malfunction in the system can be

detrimental to vehicle safety, performance, and reliability. Predicting and identifying APS failures accurately is essential for reliable operation. (Taghandiki et al., 2023; Selvi et al., 2022)

This project addresses a binary classification problem in which each instance must be classified as either an APS-related failure (“pos”) or a non-APS failure (“neg”). The challenge is that the training dataset has a severe class imbalance in which the failure instances make up less than 2% of all instances.

1.2 Dataset Description

The dataset used in this study is the APS failure at Scania Trucks dataset, which was originally provided by Scania CV AB. It was used on the Industrial Challenge at the 15th International Symposium on Intelligent Data Analysis (IDA) in 2016. The dataset contains operational measurements collected from heavy trucks during everyday usage. The training dataset has 60,000 instances, while the test dataset has 16,000 instances. Each instance has 171 anonymized numerical features representing system measurements.

A key characteristic of the dataset is its severe class imbalance. Only 1,000 of the 60,000 training instances correspond to APS failures, while the remaining 59,000 represent non-failure cases. This 1.67% imbalance presents a challenge for machine learning models that must be considered when building and training the model and evaluating its performance. Additionally, the dataset contains a substantial number of missing values, originally represented by the string “na”, which must be handled during preprocessing (IDA2016Challenge, 2016).

1.3 Research Questions

The goal of this project is to address the following questions:

1. Which machine learning model provides the best overall performance for predicting APS failures based on F1 score?
2. How does the class imbalance affect the ability of the different models to detect APS failures, particularly in terms of recall and the trade-off between false positives and false negatives?
3. What preprocessing techniques and model choices are most important for the models to have the strongest performance possible for this classification task?

1.4 Project Objectives

The objectives for this project are:

1. Perform an exploratory data analysis to better understand the dataset.
2. Conduct preprocessing to clean and prepare the dataset for the models.
3. Train and evaluate six different machine learning models.
4. Compare the models’ performances using the selected evaluation metrics.
5. Identify and justify the model that has the best overall performance for detecting failures while maintaining accuracy.

1.5 Report Structure

The remainder of this report is organized as follows. Section 2 presents the Exploratory Data Analysis, covering class and feature distributions, missing values, and correlation analysis. Section 3 outlines the methodology, including the complete preprocessing stage, feature engineering, the six machine learning models, experimental setup, hyperparameter tuning, and the evaluation metrics. Section 4 presents and discusses the experimental results for each model. Finally, Section 5 summarizes the key findings, answers the research questions, and explores potential directions for future work. References are listed at the end of the report.

2. Exploratory Data Analysis (EDA)

Before constructing any of the chosen machine learning models, a thorough Exploratory Data Analysis was conducted to understand the dataset structure and identify any limitations that may be present. The EDA examined the class distribution of the target variable, the magnitude of missing values, the distributions of individual features, and the correlation among features. The results from the EDA directly influenced the choices made during the preprocessing stage and model selection described in Section 3.

2.1 Class Distribution

One of the first steps in the EDA was examining the distribution of the target variable. The dataset suffers from an extreme class imbalance. Out of the 60,000 training instances, approximately 59,000 (98.33%) belong to the negative class (no APS failure), while only 1,000 (1.67%) belong to the positive class (APS failure).

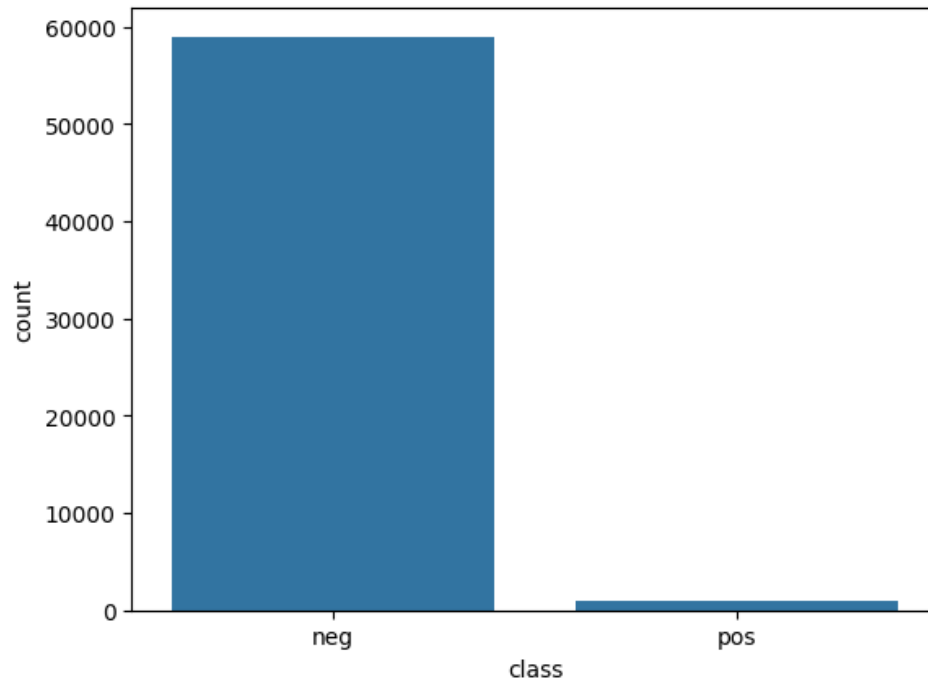


Figure 1: Distribution of APS failure in the training dataset, showing strong imbalance between the positive and the negative class.

This severe imbalance presents a major challenge: a naive classifier that always predicts “no failure” would have an accuracy of 98.33% while failing to detect any actual failures. This confirms that accuracy alone is not a sufficient evaluation metric for this problem. Models must be assessed on recall and F1-score. Additionally, class imbalance may require specific modeling strategies such as changing the weights to ensure that failure instances receive the necessary attention during training.

2.2 Missing Values

A key characteristic of the dataset is the presence of missing values. Missing datapoints are represented by the string “na” rather than the standard null representation. This feature must be converted before any analysis can happen. Building and training models on a dataset with missing values leads to a host of issues: they can create bias in the model, result in inaccurate predictions, and even lead to degradation of the model (Galli, 2023). After converting “na” values to NaN, the proportion of missing data was computed for each feature. This analysis showed that some features are missing a significant amount of data.

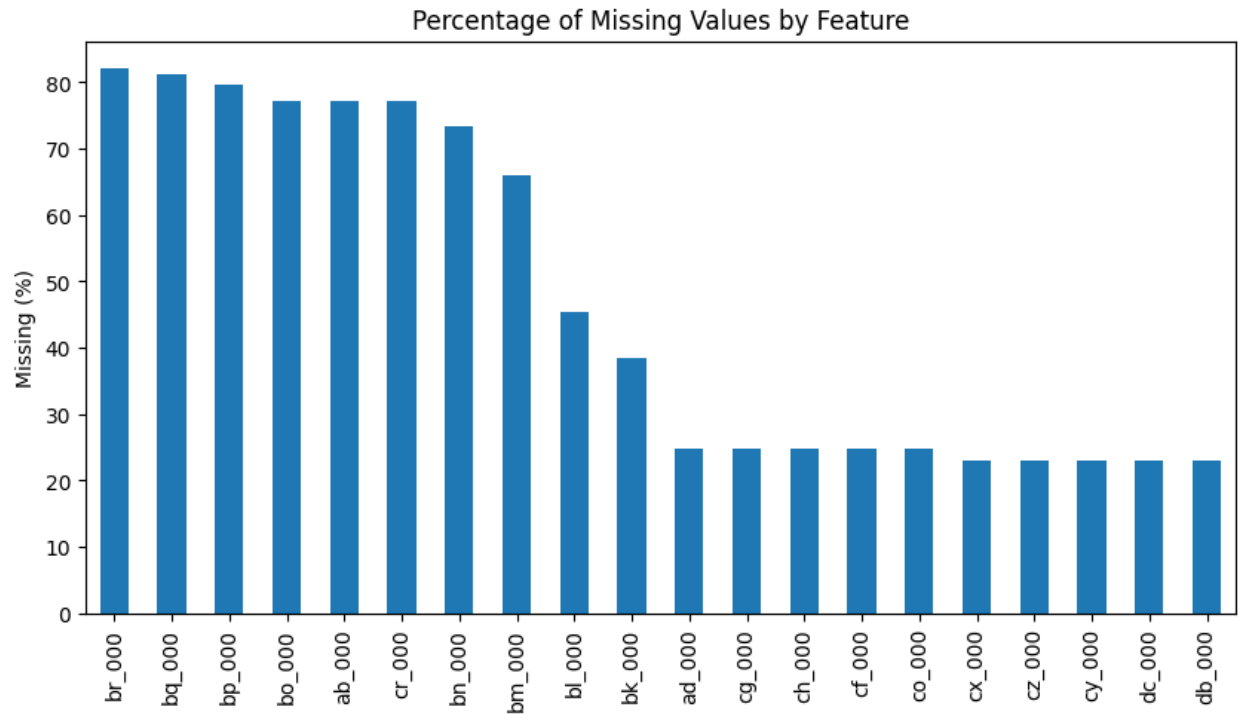


Figure 2: Percentage of missing values across the top 20 attributes. Several features exceed the 80% missing threshold and were removed during preprocessing.

Features missing more than 80% of the values were considered candidates for removal during preprocessing. These features are unlikely to contribute useful predictive information and could cause extra noise or instability for the models. For the remaining features, median imputation was applied, as discussed in Section 3.

2.3 Feature Distribution

To better understand the numerical characteristics of the dataset, histograms were created for several features, including aa_000, ac_000, ee_005, and ee_007. These features were selected to illustrate typical patterns present in the dataset.

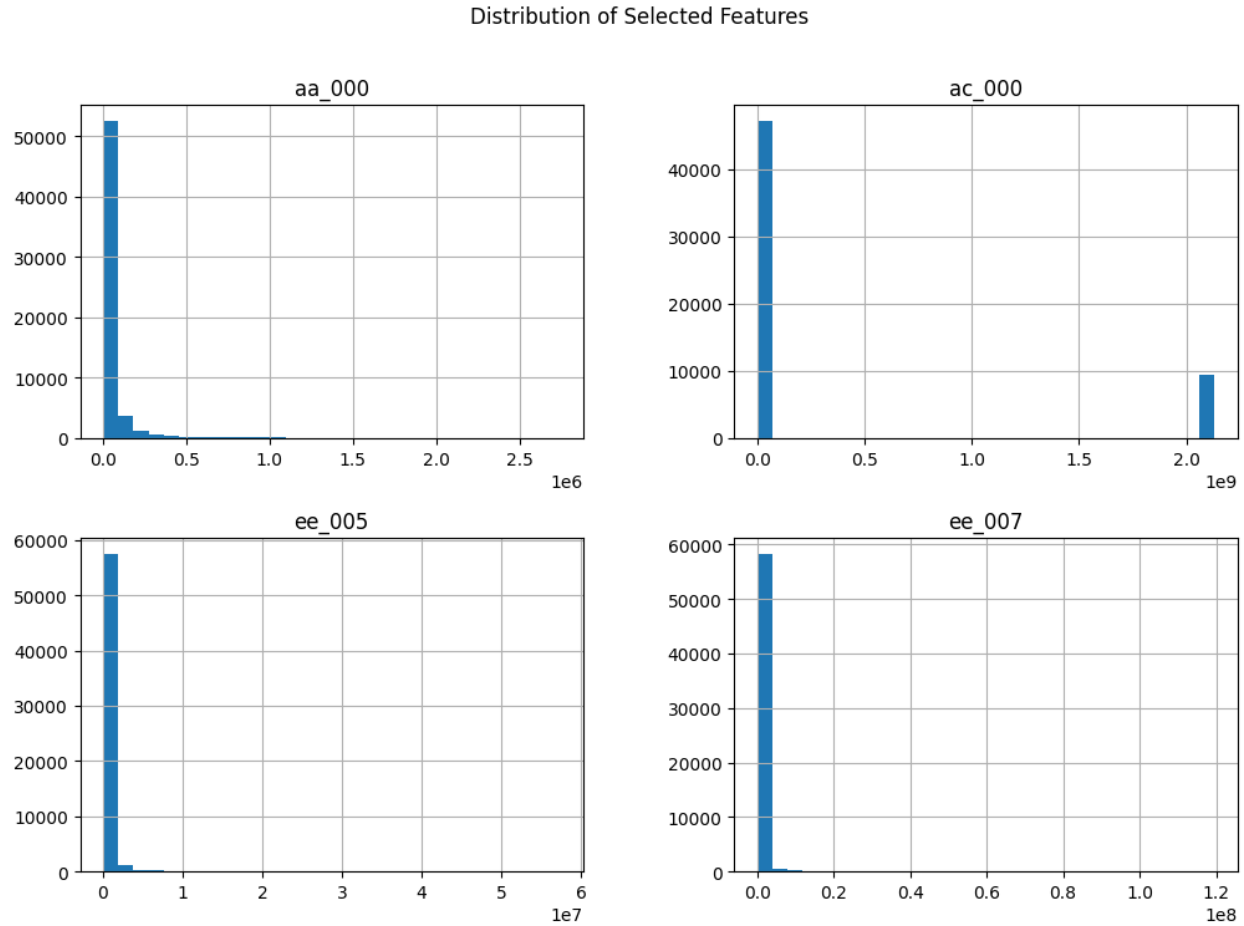


Figure 3: Histograms of selected features (`aa_000`, `ac_000`, `ee_005`, `ee_007`) showing highly right-skewed distributions and the presence of extreme outliers in the dataset.

The histograms show that most numerical features are highly right-skewed, with the majority of observations concentrated near zero or low values, and a small number of extreme values.

This kind of distribution is important to understand before building the models. Linear models such as Logistic Regression and distance-based models like KNN are sensitive to the distribution and scale of the features it receives. Extremely skewed features can negatively impact the model weights and distance calculations, leading to inaccuracy in the model. This observation is the reason why a logarithmic transformation and standardization were done during preprocessing.

2.4 Correlation Analysis

Correlation analysis was performed to examine potential relationships between features. A correlation heatmap was created for a subset of features to identify highly correlated values.

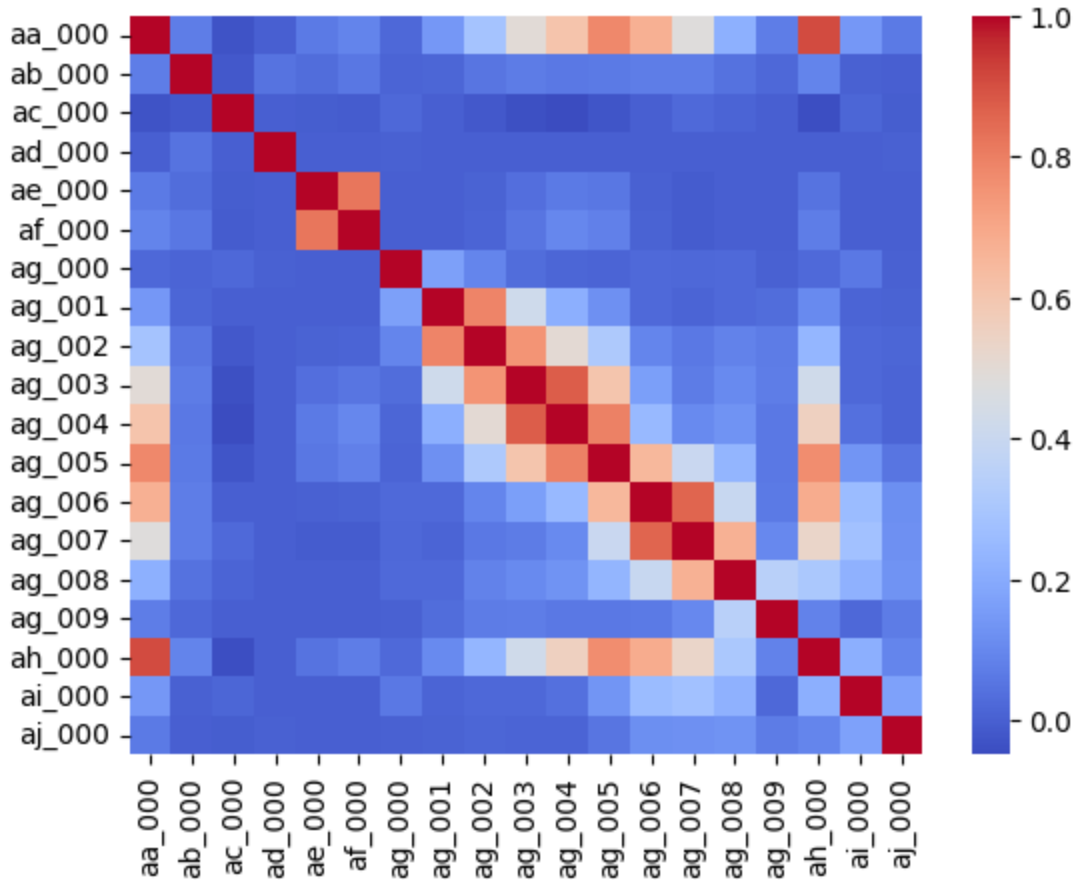


Figure 4: Correlation heatmap for a subset of numerical features, showing some correlations between features.

The heatmap shows that some features share positive correlations, suggesting that they may represent related measurements. However, the overall correlation is very complex because of the large number of anonymized features in the dataset. Understanding these relationships is important when doing feature engineering and choosing models in the later stages of this project.

2.5 Summary of Key Insights

The exploratory data analysis showed important characteristics of the dataset that influenced the modeling approach:

- The dataset is extremely imbalanced, with significantly fewer APS failure instances compared to non-failure instances.
- Missing values are present across many features and require some attention during preprocessing.
- Several features are very skewed and contain extreme values.
- Dimensionality reduction and scaling features should be considered due to the large number of numerical attributes in the dataset.

These observations determined what steps were necessary to do during preprocessing and which models were chosen, as described in the following section.

3. Methodology

This section describes the complete methodology in this project, including data preprocessing, feature engineering, the six chosen machine learning models, the experiment setup, hyperparameter tuning, and the evaluation metrics used to assess the models' performance.

3.1 Data Preprocessing

Before training any machine learning models, several preprocessing steps were taken to clean and prepare the dataset. These steps addressed the issues that were identified during the exploratory data analysis including missing values, inconsistent data formats, and feature scaling.

3.1.1 Missing Values

The missing values in the dataset were originally represented as the string "na" rather than the standard null value. The first step was to replace all occurrences of "na" with the NumPy NaN value using the pandas `replace()` method.

3.1.2 Data Type Conversion

Because the missing values were stored as strings, many of the columns were seen as objects rather than numeric data types. After the NaN conversion, all columns were cast to numeric types using `pd.to_numeric()` with `errors='ignore'`, keeping the target class columns as a categorical variable.

3.1.3 Removal of Features Missing Significant Data

After computing the proportion of missing values for each feature, it was determined that some features were missing more than 80% of values. These features were identified and removed from both the training and test sets. The 80% threshold was chosen because features missing such a significant amount of data are unlikely to contribute anything useful to the models and were more likely to cause problems.

3.1.4 Median Imputation

For the remaining features, missing values were imputed using the median of each column, calculated from the training set only. Median imputation was chosen instead of mean

imputation because the extreme unbalance of the dataset and presence of outliers could cause the mean to be skewed towards the majority class distribution.

3.1.5 Logarithmic Transformation

To address the severe right-skewness observed in the EDA, a logarithmic transformation was applied to both the training and test datasets. The reason for this was to compress the large values, reduce the impact the extreme outliers have on the dataset, and overall make the features more symmetric. Log1p was chosen instead of just log() to handle the numerous zero values present in the dataset.

3.1.6 Feature Standardization

After the logarithmic transformation, all features were standardized using scikit-learn's StandardScaler, which transforms the features to have a mean of zero and a standard deviation of 1 (scikit-learn, 2019). Standardization is extremely important for distance-based models like KNN and SVM, and linear models like Logistic Regression. The scaler was fitted only on the training dataset and then was applied to the test dataset to avoid leaking any information.

These preprocessing steps resulted in a clean and standardized dataset, ready to be used for developing and testing the models.

3.2 Feature Engineering and Selection

No new features were explicitly created in this project. However, feature selection was implicitly performed by removing features with a high proportion of missing values, as described in Section 3.1.3.

Principal Component Analysis (PCA) was used to reduce the dimensionality of the dataset and for visualization purposes, particularly for the SVM decision boundary plot. However, PCA was not applied to the main dataset for training the models. This is because models like Random Forest and XGBoost do not need reduced features, and applying PCA would not be beneficial for the models.

3.3 Machine Learning Models

- The machine learning models chosen for this project include:
- Logistic Regression: A supervised model used as a baseline for binary classification. It predicts the probability that an instance belongs to a specific class. (GeeksforGeeks, 2017)
- Decision Tree: A supervised model that can be used for classification problems by splitting the data into different branches that show the possible outcomes (Mihindukulasuriya, 2025).

- K-Nearest Neighbors: A non-parametric, supervised model. It is distance-based and makes classifications and predictions about a data point based on the closest data points it is surrounded by (IBM, 2021).
- Random Forest: An ensemble model that combines many decision trees to reduce overfitting and improve predictions (GeeksforGeeks, 2024).
- Support Vector Machine: A supervised model, typically used for classification problems, that finds the maximum distance between classes, also known as the decision boundary (IBM, 2023).
- XGBoost: A gradient boosting model that combines weak models to create a strong model by building decision trees sequentially. Each new tree corrects the errors of the previous tree (GeeksforGeeks, 2021).

3.3.1 Logistic Regression

Logistic Regression was chosen as a baseline model because of its simplicity and suitability for binary classification problems. The goal of logistic regression is to visualize the probability that an instance belongs to one of two classes. It is appropriate for the APS failure prediction problem since the target variable consists of two classes: failure and non-failure.

Because the dataset is extremely unbalanced, class weights were adjusted using `class_weight='balanced'` to ensure that the APS failure class received more attention during training. Feature scaling was applied prior to training, as Logistic Regression is sensitive to the magnitude of the features. Hyperparameter tuning was done using GridSearchCV with 5-fold cross-validation, where the parameter C was tested with multiple values. The F1-score was used at the primary evaluation metric because it balances precision and recall.

3.3.2 Decision Tree

Decision Tree was chosen for this project because of its efficiency in solving binary classification problems. However, decision trees are prone to overfitting, especially with large and complicated datasets like the APS datasets. To resolve this, pre-pruning was applied by tuning parameters such as the maximum depth and minimum samples per leaf. GridSearchCV was used to optimize the chosen hyperparameters using 5-fold cross-validation, with F1-score as the evaluation metric.

3.3.3 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a “...non-parametric, supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual point” (IBM, 2021). KNN does not assume anything about a data point but compares it to its closest neighbors, determined by the parameter k, and then classifies it accordingly. Because KNN is based on distance calculations, scaling the features was extremely important to make sure that all features contributed equally to the model and there was no bias. Without scaling, larger features would dominate the distance calculations and would result in inaccurate predictions.

GridSearchCV was used for hyperparameter tuning with 5-fold cross-validation. The parameters that were tuned include the number of neighbors (k), distance metric (Euclidean vs. Manhattan), and the weights (uniform vs. distance-based). Distance-weighted neighbors were particularly useful, as they gave greater importance to closer datapoints, improving the classification of minority class instances.

3.3.4 Support Vector Machine (SVM)

The SVM model was implemented using a Radial Basis Function (RBF) kernel to find the nonlinear relationships in the dataset. The use of an RBF kernel enables the model to create the decision boundaries and separate the features of the dataset. SVM is particularly good for this classification problem because it works by identifying the best decision boundary that maximizes the margin between classes (IBM, 2023).

The class imbalance was handled by using `class_weight='balanced'`, and the features were scaled prior to training. Hyperparameters such as `C` and the kernel coefficient (gamma) were tuned using GridSearchCV with 3-fold cross-validation. 3-fold was used instead of 5-fold like the other models to reduce the training time, as SVM training is more exhaustive than the other models on these datasets.

3.3.5 Random Forest

Random Forest was chosen as one of the ensemble learning models because of its capability for classification tasks, especially for complex datasets. RF works by creating multiple decision trees during training and later combining them based on a majority vote by the other trees. The reason it works so well for this project is because the random features for each tree prevents the model from overfitting, which it is prone to, and makes the overall prediction more accurate.

3.3.6 XGBoost

XGBoost was chosen because of its performance on imbalanced datasets. It works by building trees sequentially so that each new tree corrects the errors of the previous trees. The `scale_pos_weight` parameter was adjusted so that the minority class received enough attention during training. The hyperparameters were adjusted- `n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, and `colsample_bytree=0.8`. These were adjusted based on the results of the GridSearchCV, which gave the best parameters for the model.

3.4 Experimental Setup

The APS dataset provided was already split into training and test sets. All steps taking during preprocessing, including handling the missing values, feature transformation, and scaling, were applied only to the training dataset and then to the test dataset to prevent data leakage. This

ensured that the models didn't just memorize the training data and gave them a fair evaluation with the test dataset.

All models were implemented in Python using the libraries pandas and NumPy for data manipulation; scikit-learn for preprocessing, model training, hyperparameter tuning, and evaluation; XGBoost for the gradient boosting model; and Matplotlib and Seaborn for visualization. GridSearchCV from scikit-learn was used for hyperparameter tuning with cross-validation. The target variable was encoded as 0 for the negative class (no APS failure) and 1 for the positive class (APS failure).

3.5 Hyperparameter Tuning

Hyperparameter tuning was performed on all six models using GridSearchCV, which performs a search over a specified parameter grid using k-fold cross-validation. The F1-score was used as the optimization metric for all models. Table 1 summarizes the hyperparameters tuned for each model and the best values found.

Model	Parameters Tuned	Search Range	Best Parameters
Logistic Regression	C (regularization)	{0.01, 0.1, 1, 10}	C = 0.1
Decision Tree	max_depth, min_samples_leaf, class_weight	{5, 10, 15, None}; {5, 10, 20}; {None, balanced}	depth=10, leaf=5, weight=None
KNN	n_neighbors, weights, metric	{3, 5, 7, 11}; {uniform, distance}; {euclidean, manhattan};	k=3, distance, manhattan
SVM	C, gamma	{1, 10}; {scale, 0.1}	C=10, gamma=scale
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf, class_weight	{100, 200}; {None, 10, 20}; {2, 5}; {1, 2}; {None, balanced}	200 trees, depth=None, split=2, leaf=1
XGBoost	fixed configuration with scale_pos_weight	scale_pos_weight= neg/pos ratio	scale_pos_weight=59

Table 1: Hyperparameter tuning details for all six models.

3.6 Evaluation Metrics

Several evaluation metrics were used to assess the results from each model, especially because of the significant class imbalance:

- **Accuracy** measures the overall proportion of correctly classified instances. However, it is unreliable for imbalanced datasets and cannot be the only metric used to evaluate the models.
- **Precision** measures the proportion of predicted positive cases that are actually positive. The higher the precision score, the fewer the false positives that were classified by the model.
- **Recall** measures the proportion of actual positive cases that are correctly identified. This metric is important for this project, as failing to detect APS failures can have serious consequences.
- **F1-score** is the mean of the precision and recall scores, giving a balanced measure of performance. (GeeksforGeeks, 2020)

Due to the imbalanced nature of the dataset, evaluation metrics such as Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE), which are typically used for regression problems, were not used for this classification problem. Instead, the results from the models were evaluated using precision, recall, F1-score, and confusion matrices. F1-score was emphasized because of the balance it gives between recall and precision.

4. Results and Discussion

This section presents the results from all six trained models on the test set of 16,000 instances. Results are stated for accuracy, precision, recall, and F1-score. A confusion is provided for each model, along with visualization of the Decision Tree model and the SVM decision boundary.

4.1 Logistic Regression Results

The Logistic Regression model that was trained with balanced class weights and the hyperparameter $C=0.1$ achieved an overall accuracy of 96.45%. Its recall score of 0.95 was the highest of models and means that the model successfully classified 94.93% of actual APS failures in the test set. However, the model had a precision score of 0.39, meaning that 39% of instances predicted as failures were actual failures, which resulted in 549 false positives.

The confusion matrix shows this trade-off with a large number of false positives but very few false negatives. While the model is good at detecting failures, it struggles to distinguish between failure and non-failure cases.

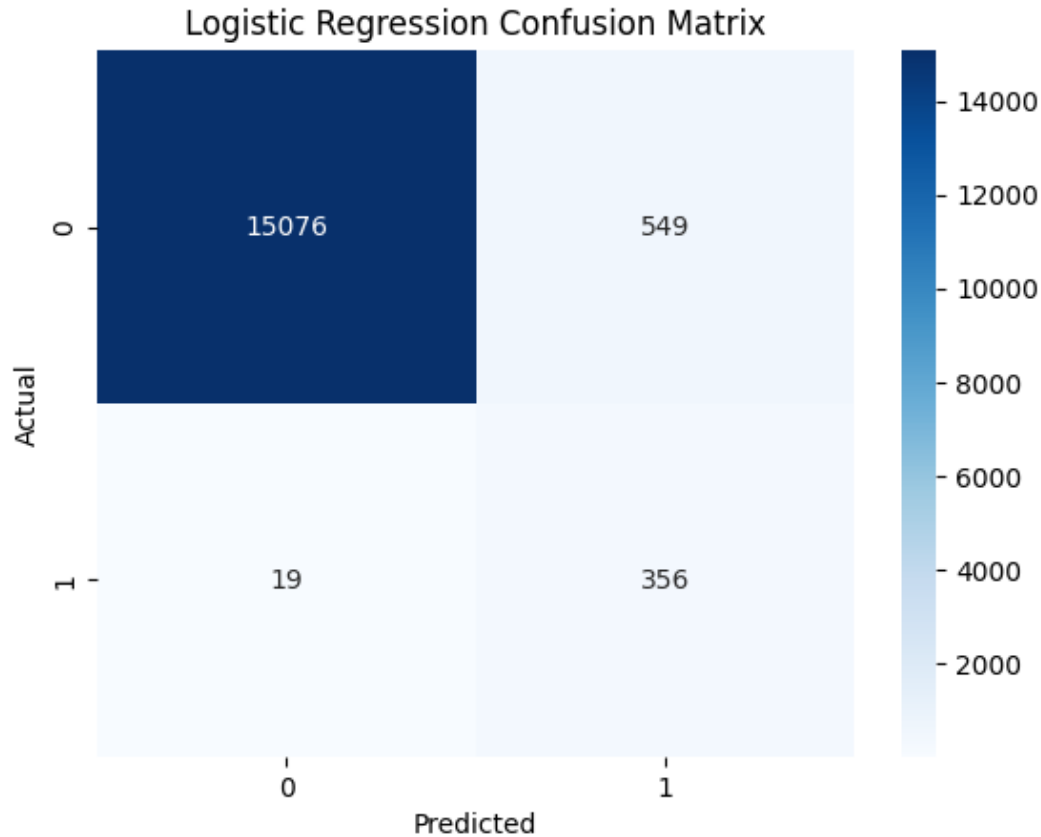


Figure 5: Confusion matrix for Logistic Regression (TP=356, FP=549, FN=19, TN=15,076). The high number of false positives, 549, compared to the true positives, 356, shows the model's tendency to over-predict failures when using balanced class weights.

As shown in Figure 5, the confusion matrix indicates that the model correctly classified 15,076 non-failure instances and 356 failure cases. There were 549 false positives and 19 false negatives, meaning some actual failures were not identified. Despite the high recall score, the F1-score of 0.5563 ranks Logistic Regression last among all models.

4.2 Decision Tree Results

The tuned Decision Tree Model (max_depth=10, min_samples_leaf=5, class_weight=None) achieved an accuracy score of 98.86%, precision of 0.87, recall of 0.60, and F1-score of 0.71. While the model is relatively precise when it predicts a failure, its recall is the lowest among all the non-linear models, meaning it misses nearly 40% of actual failures.

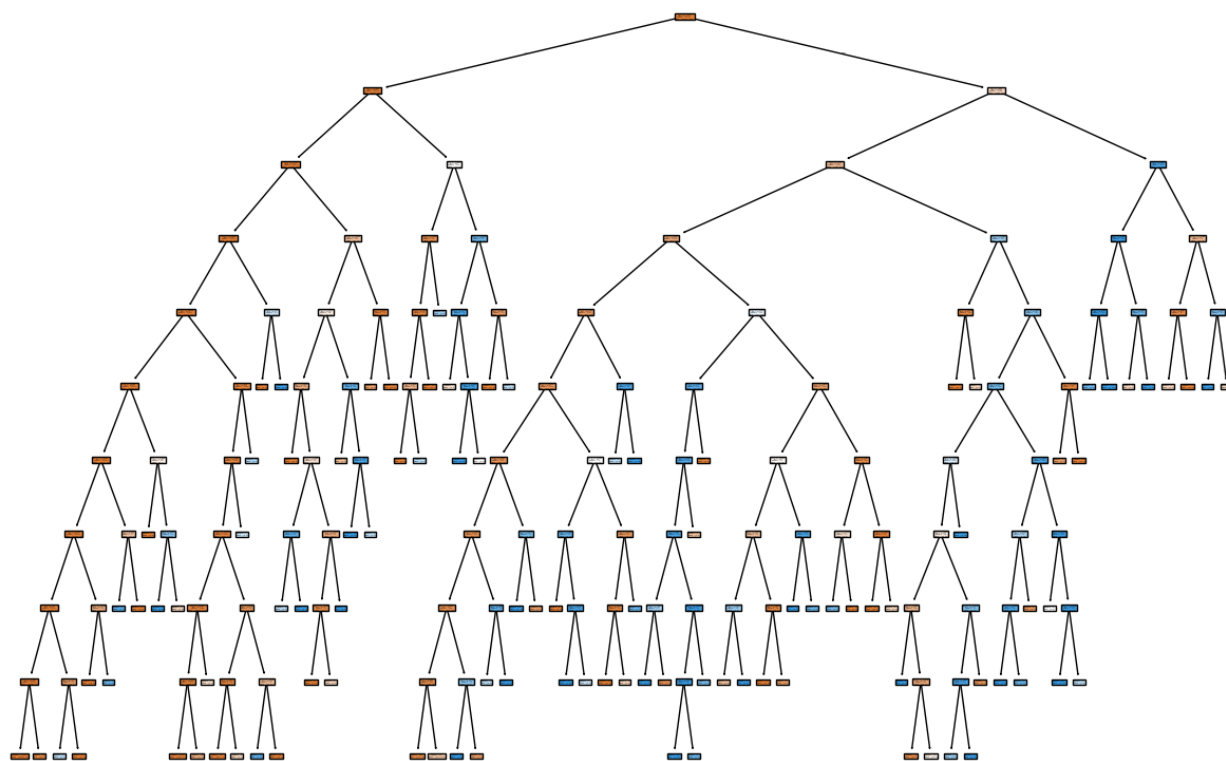


Figure 6: Visualization of the tuned Decision Tree structure. Orange nodes represent the negative class, and the blue nodes represent the positive class.

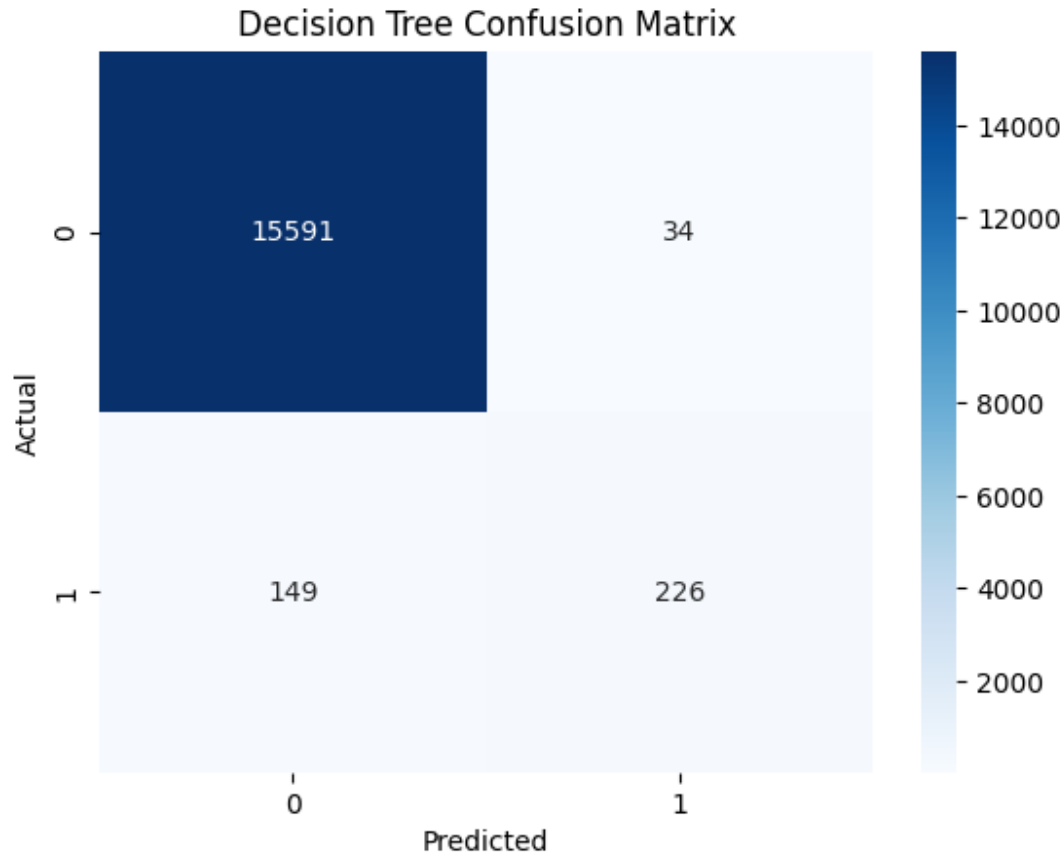


Figure 7: Confusion matrix for Decision Tree (TP=226, FP=34, FN=149, TN=15,591). While false positives are very low at 34, the 149 false negatives show a significant number of missed failures.

As shown in Figure 6, the limit set on the depth prevents the model from overfitting to the training data, but that may also prevent it from learning the difference between failure and non-failure instances. The confusion matrix shows that there were 149 false negatives; the model may be conservative in predicting the positive class because of the imbalanced training data.

4.3 K-Nearest Neighbors (KNN) Results

The tuned KNN model ($k=3$, distance-weighted, Manhattan metric) had an accuracy of 99.08%, precision of 0.91, recall of 0.67, and an F1-score of 0.77. The use of distance-weighted and the Manhattan metric improved the model's ability to detect instances of the minority classes compared to the baseline KNN model that was tested.

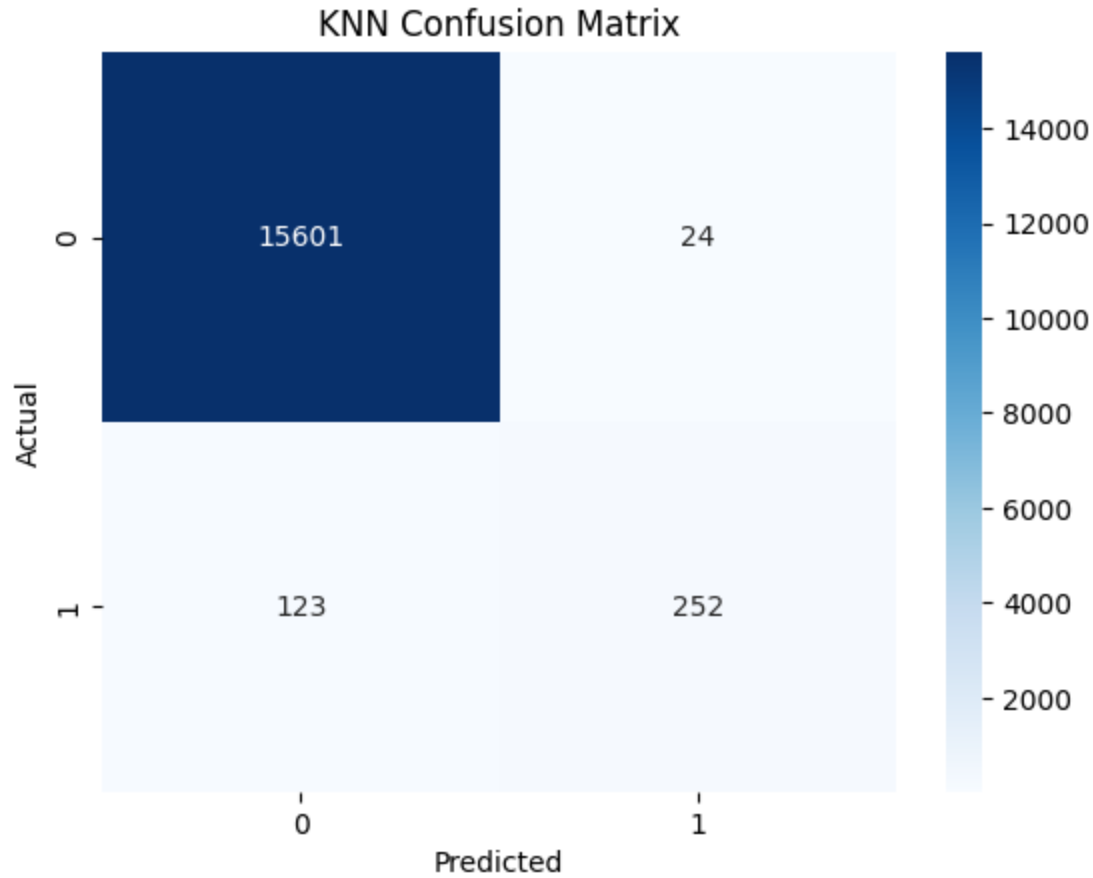


Figure 8: Confusion matrix for K-Nearest Neighbor (TP=252, FP=24, FN=123, TN=15,601). The model has only 24 false positives, but missed 123 actual failures.

The confusion matrix shows that the model correctly classified 15,601 non-failure instances and 252 failure instances, while producing 24 false positives and 123 false negatives. Compared to the initial KNN model, changing the hyperparameters significantly reduced the number of missed failures (162 to 123), improving the model's ability to detect APS failures.

4.4 Support Vector Machine (SVM) Results

The SVM with RBF kernel ($C=10$, $\gamma=\text{scale}$, balanced class weights) had an accuracy of 99.07%, precision of 0.85, recall of 0.73, and F1-score of 0.79. By maximizing the margin between classes, the SVM model had a better recall than the KNN and Decision Tree models while still maintaining a reasonable precision score.

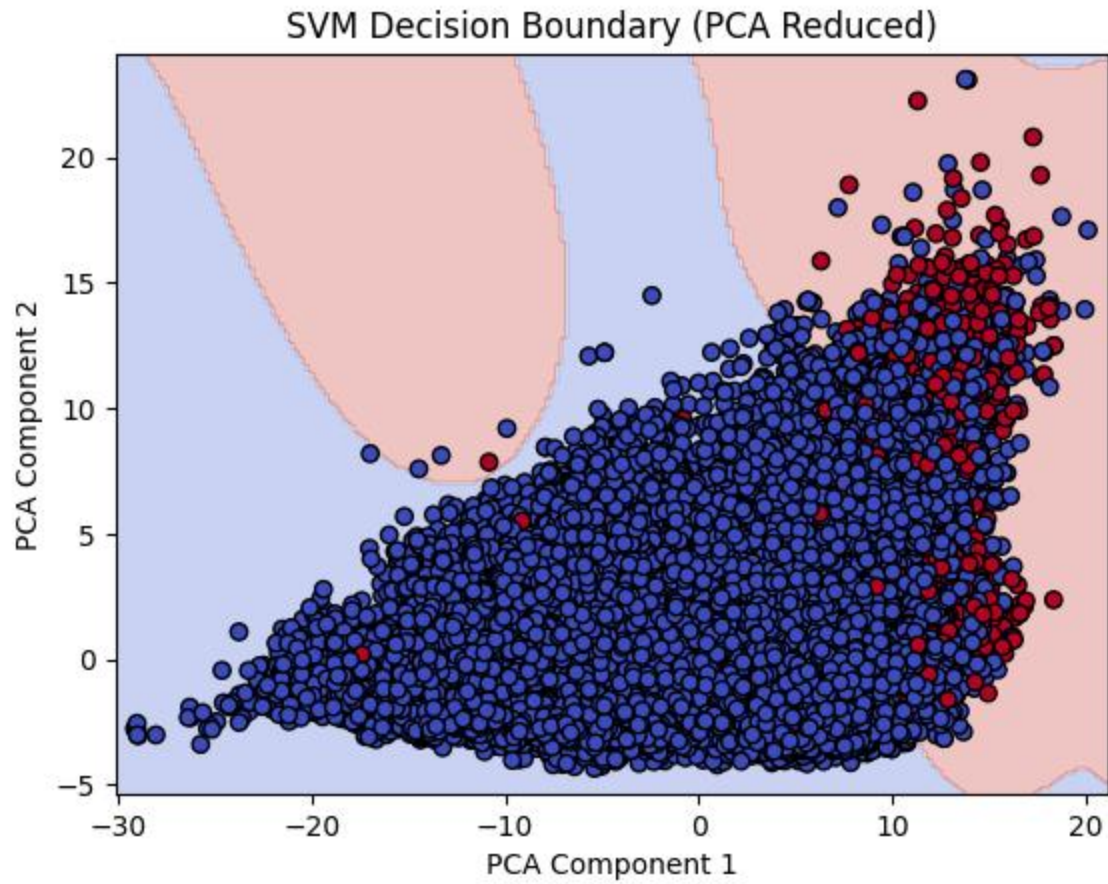


Figure 9: SVM decision boundary visualized in PCA-reduced 2D space. Blue points represent the non-failure instances, and the red points represent APS failures. The pink area shows the decision region for the positive class.

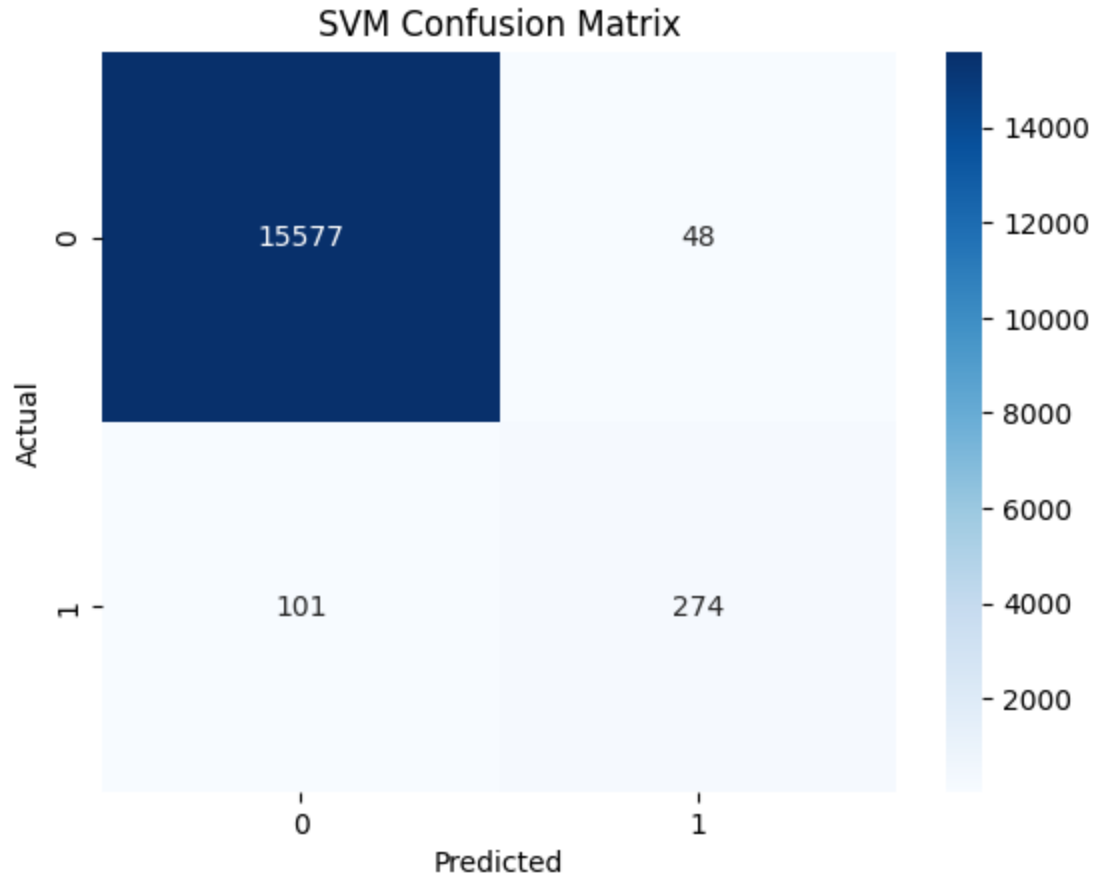


Figure 10: Confusion matrix for SVM (TP=274, FP=48, FN=101, TN=15,577). The SVM has a better recall than KNN and Decision Tree.

The use of the RBF kernel for SVM allows the model to create non-linear decision boundaries which separates the failure class from the non-failure class in a more efficient manner than a linear classifier. The PCA-reduced decision boundary plot shows that the SVM model effectively separated the two classes.

4.5 Random Forest Results

The tuned Random Forest model had an accuracy of 99.21%, a recall of 0.7093, a precision of 0.9399, and an F1-score of 0.8085, which was the second best F1-score of all the models.

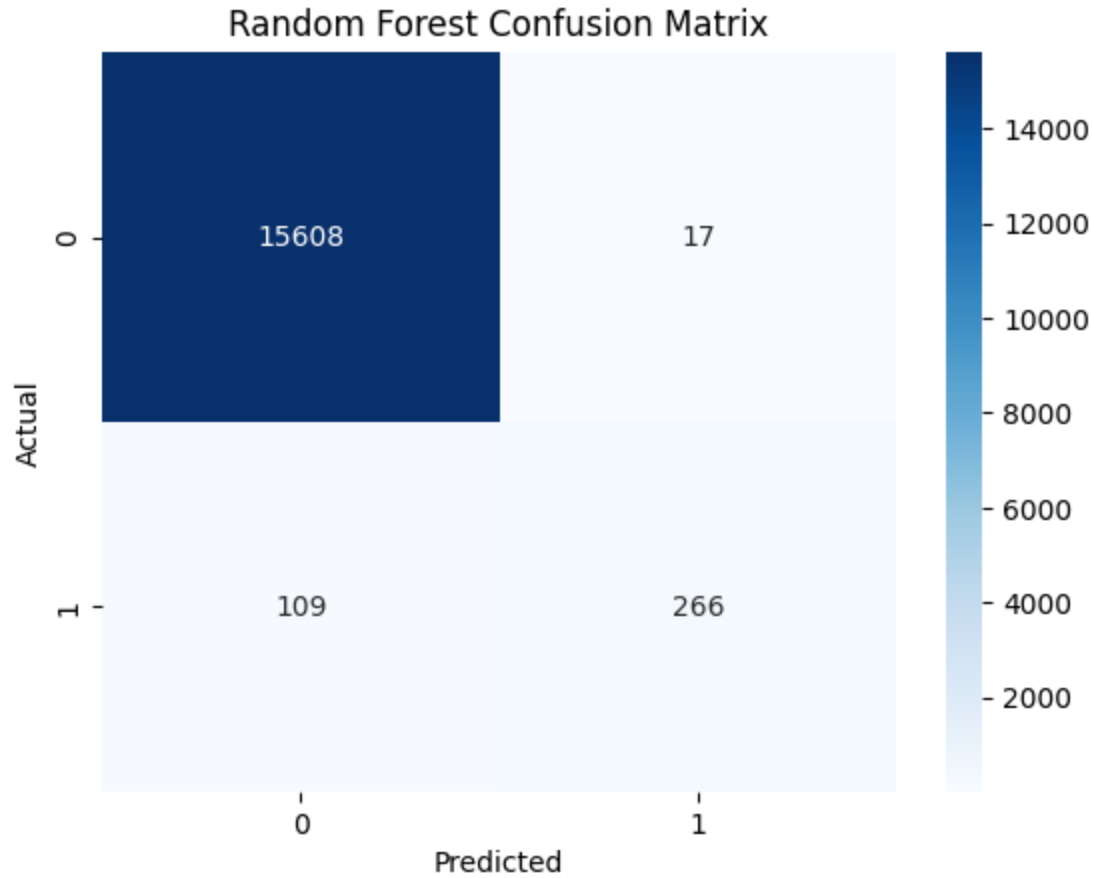


Figure 11: Confusion matrix for Random Forest (TP=266, FP=17, FN=109, TN=15,608). This model had the lowest false positive count of 17, showing its high precision of 0.94.

The tuned Random Forest model had the second-best performance of all six models. It had the highest precision of the models, meaning that when it predicted an APS failure, it was right over 93% of the time. However, the recall score of 0.7093 means that it missed some actual failures. The feature importance shown in Figure 12 provides some insightful information. It shows that features 25, 8, and 70 overrule the other features, having scores much higher than the features that follow. Likely this means these features have the most important information about the health of the APS system.

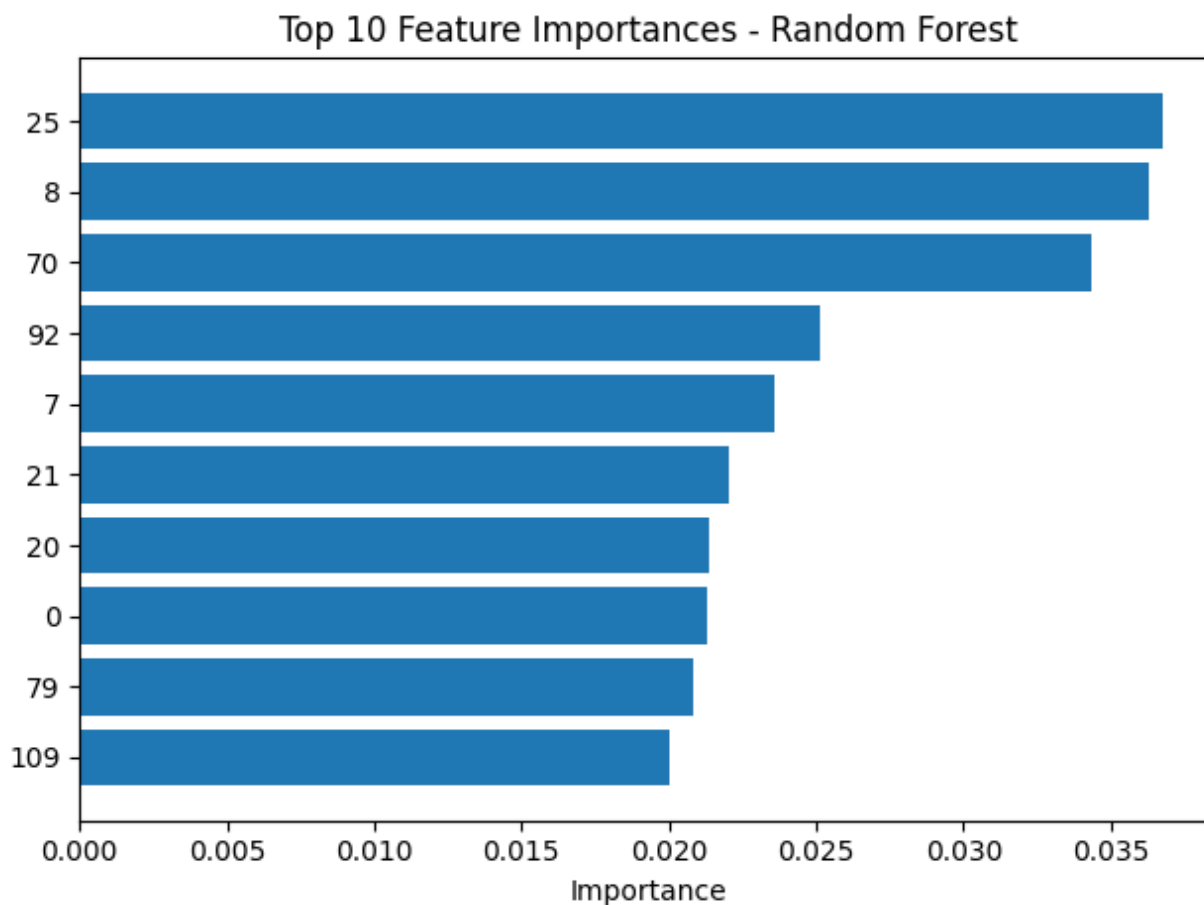


Figure 12: Top 10 feature importances for Random Forest. Features 25, 8, and 70 are the sensor measurements that are strongest predictors in the Random Forest model. The features are identified by index due to the anonymized dataset.

4.6 XGBoost Results

The XGBoost model had the best overall performance of all the models, with an accuracy of 99.35%, a precision of 0.8712, a recall of 0.8480, and an F1-score of 0.8595. Unlike the other models, XGBoost had a strong balance between the precision and recall scores.

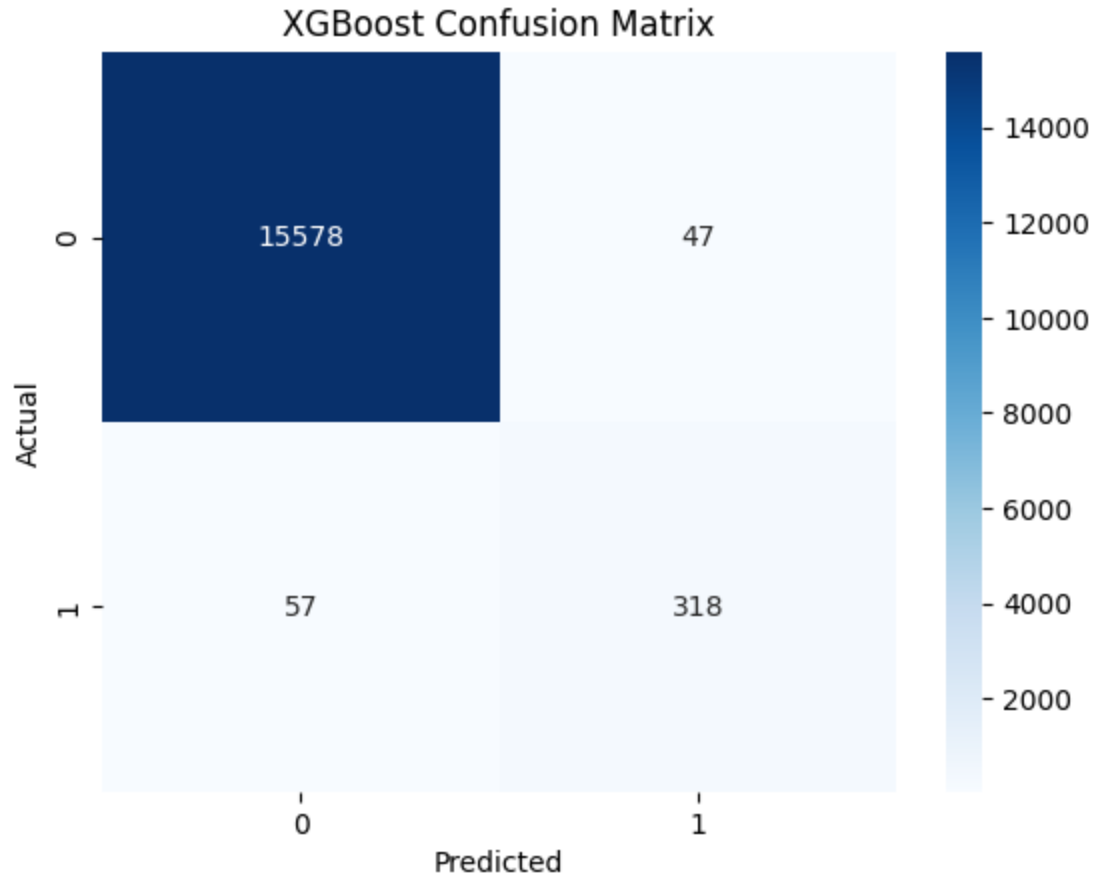


Figure 13: Confusion matrix for XGBoost (TP=318, FP=47, FN=57, TN=15,578). XGBoost had the fewest number of false negatives, 57.

In APS failure prediction, a false negative means that a real failure goes undetected, which can result in a vehicle malfunction or an expensive repair. By minimizing the false negatives while maintaining a strong precision, XGBoost has the best balance which makes it the model reliable model for this particular classification task.

4.7 Results Tables

Table 2. Training Results

	Model	Accuracy	Precision	Recall	F1
5	XGBoost	0.99850	0.991080	1.000	0.995520
4	Random Forest	1.000000	1.000000	1.000	1.000000
3	SVM	0.998017	0.894360	0.999	0.943788
2	KNN	1.000000	1.000000	1.000	1.000000
1	Decision Tree	0.995333	0.947761	0.762	0.844789
0	Logistic Regression	0.962933	0.306206	0.967	0.465127

Table 3: Testing Results

	Model	Accuracy	Precision	Recall	F1-score
5	XGBoost	0.993500	0.871233	0.848000	0.859459
4	Random Forest	0.992125	0.939929	0.709333	0.808511
3	SVM	0.990688	0.850932	0.730667	0.786227
2	KNN	0.990812	0.913043	0.672000	0.774194
1	Decision Tree	0.988563	0.869231	0.602667	0.711811
0	Logistic Regression	0.964500	0.393370	0.949333	0.556250

Table 2 provides the results of all six models when they were run on the training dataset, and Table 3 provides the results of all six models after they were run on the test dataset, both ordered by F1-score.

XGBoost had the highest test F1-score of 0.8595, meaning that it had the best overall balance between precision and recall. It's training F1-score of 0.9955 is good, but not perfect, which means that the model did not memorize the training data. This is extremely important to give us a good idea of what the model can actually do on a real dataset.

The Random Forest models also performed well, with F1-scores of 0.8085 and 0.7862, respectively. Notably, Random Forest had a perfect training F1-score of 1.0, which means that the model was overfitted on the dataset. This is because there was no limit set on the depth and so the trees grew and grew until each instance was classified. Once the hyperparameters were tuned, the model was able to generalize well on the test dataset. KNN also had a perfect training F1-score of 1.0 but the F1-score on the test dataset was 0.7742, after changing the number of k neighbors and some of the other hyperparameters.

The model with the best difference between the training and test sets was Decision Tree, which had a training F1-score of 0.8448 and a test F1-score of 0.7118. This is because of the pre-pruning that was done before running the model. The tree depth was limited to 10 to prevent it from growing so much that it just memorized the test dataset. The model missed a lot of actual failures (149), but the fact that it had a test precision score of 0.8692 was one of the highest of all models.

Logistic Regression had a training F1-score of 0.4651 and a training F1-score of 0.5546, which are both extremely low. Instead of overfitting like Random Forest or KNN, the model was underfitting on the dataset. Even though it had a high test recall score of 0.9547, it had a test precision score of only 0.3908, meaning that it classified a large number of false positives. In a real-world application, such as the Scania trucks, this would cause the perfectly fine trucks to be inspected when they don't need to be.

The difference between the training and test results helps us to understand the importance of cleaning the dataset and give the models a better performance. These results also show a key trade-off in classification problems with imbalanced data- models that prioritize recall often have lower precision scores, and vice versa. Therefore, the F1-score provides the most reliable measure of performance for this specific problem.

4.8 Final Charts/Graphs

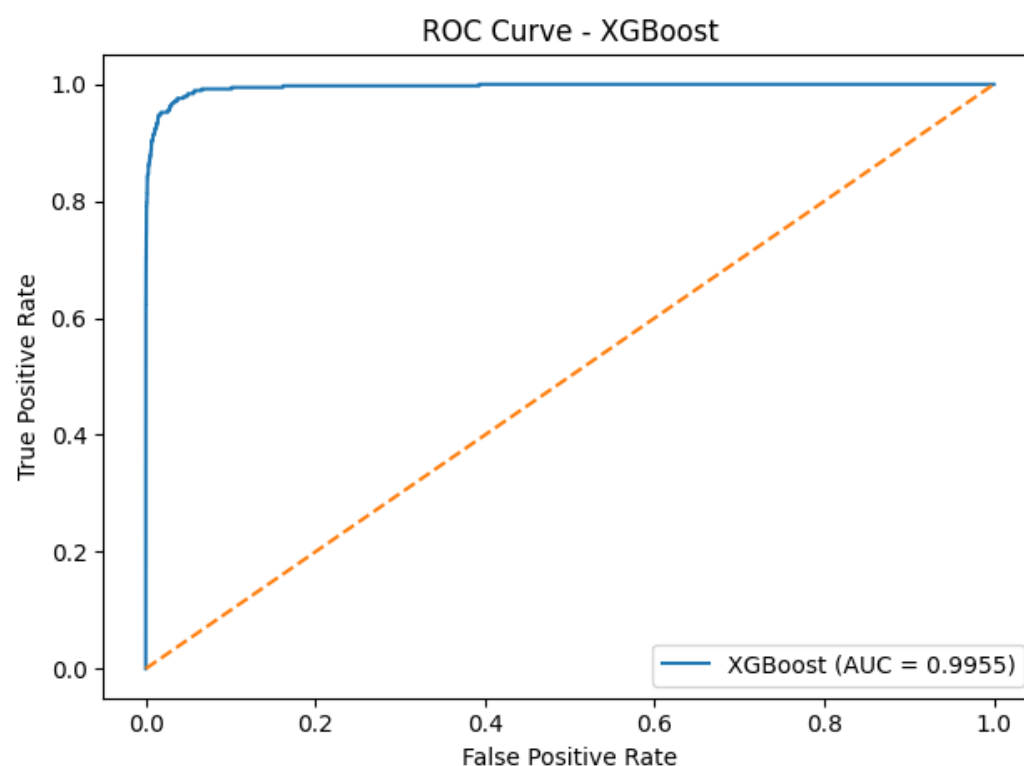


Figure 14: ROC Curve for XGBoost

The ROC curve shows the trade-off between the rate of true positives and the rate of false positives for the XGBoost model for different classification thresholds. The proximity of the curve to the top-left corner means that the model is good at separating the classes and can handle imbalanced data well.

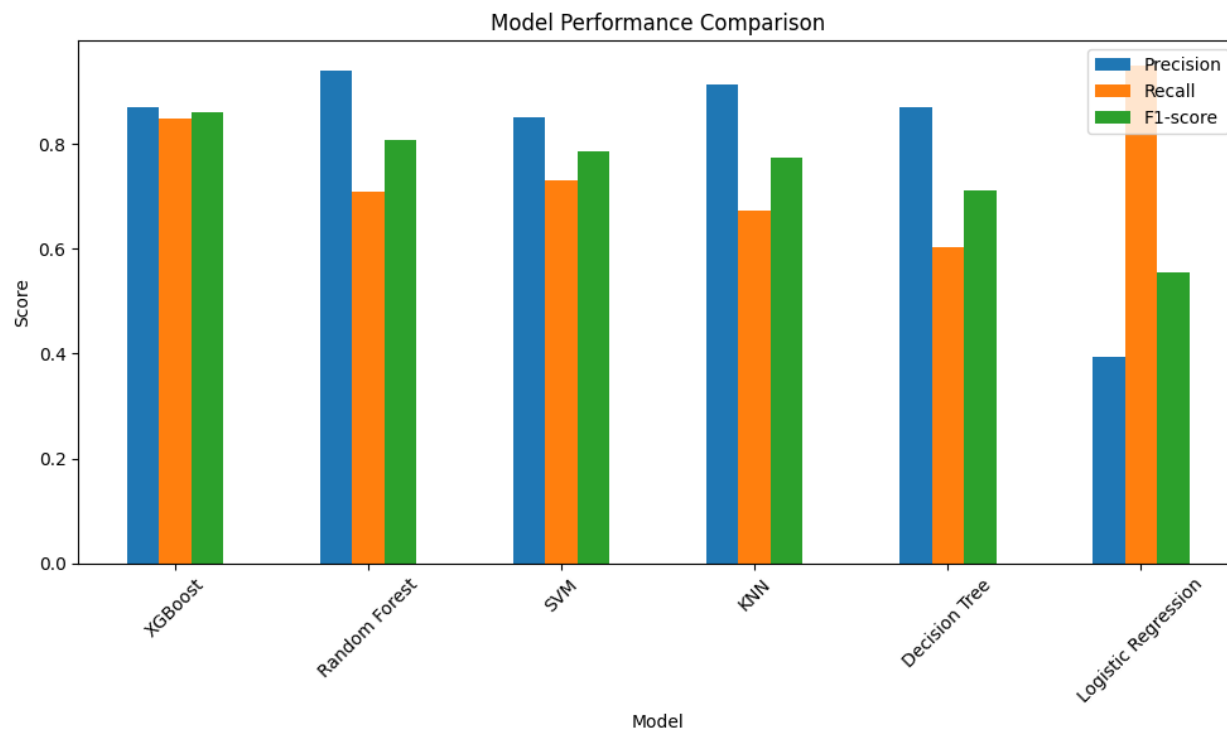


Figure 15: Model performance comparison chart for all six models

This bar chart shows a side-by-side comparison of all six models based on the chosen evaluation metrics- precision, recall, and F1-score. The model is organized so that the highest-scoring model is on the left, and all the rest of the models following in order so that Logistic Regression, the lowest-scoring model, is at the end. XGBoost has the best overall balance among all models, while Logistic Regression has an extreme contrast between the very high recall and very low precision.

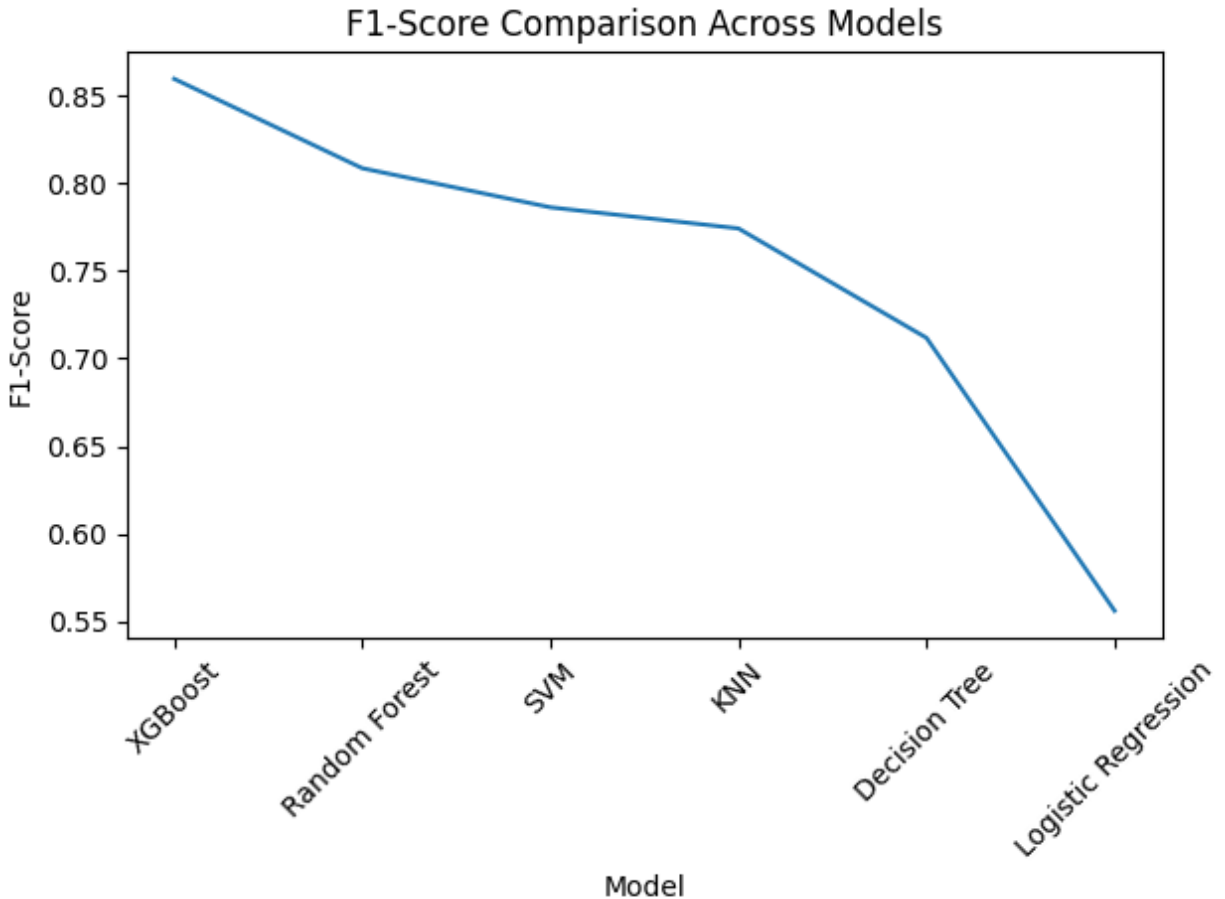


Figure 16: F1-Score comparison chart for all six models, ordered from highest to lowest. XGBoost has the highest F1 score of 0.8595, followed by Random Forest with 0.7862. Logistic Regression is at the end with 0.5563.

What this graph shows is that having a high recall score does not automatically mean that the model has a good overall performance. This is the reason why the F1-score was emphasized so heavily; it gives the best balance and analysis of the model's performance on the dataset.

5. Conclusion

5.1 Summary of Key Findings

In this project, six machine learning models were chosen for a binary classification task of predicting Air Pressure System (APS) failures in Scania trucks. The datasets were a training set of 60,000 instances and a test set of 16,000 instances with 171 anonymized sensor features. The key challenge was the severe class imbalance, since APS failures only made up 1.67% of instances in the training set.

A thorough exploratory data analysis and preprocessing steps were conducted, including features missing more than 80% of data, median imputation, logarithmic transformation, and standardization. Six models- Logistic Regression, Decision Tree, KNN, SVM, Random Forest, and XGBoost were trained, tuned, and evaluated based on the metrics precision, recall, accuracy, and F1-score.

XGBoost had the best overall performance with an F1-score of 0.8595 and a recall of 0.848, followed by Random Forest which had an F1-score of 0.8085 and a recall of 0.7093. Logistic Regression was the worst of all models, with an F1-score of 0.5563 and a recall of 0.9493. This means that gradient boosting models are the best for an imbalanced classification task.

5.2 Answers to Research Questions

Research Question 1: Which model provides the best overall performance? XGBoost had the highest F1-score of 0.8595 and the best balance of precision and recall, making it the most suitable model for the APS failure prediction task.

Research Question 2: How does the severe class imbalance affect model performance? The class imbalance was a big challenge as it affects things like recall for all of the models. Without adjusting the class weights, the models are biased towards the majority class. Changing the class weights for models like Logistic Regression and SVM, and `scale_pos_weight` for XGBoost was important to ensure that the models saw and considered the minority class.

Research Question 3: What preprocessing steps and modeling techniques are most important? Median imputation, logarithmic transformation, and standardization were the most important things that were dealt with during the preprocessing stage. These impacted the model's ability to accurately classify the instances.

5.3 Limitations

A limitation in this project was that while GridSearchCV was used to tune the hyperparameters, the parameter grids were constrained by computational resources. A more exhaustive search might show better results for the models. Second, the models were evaluated on one dataset. Having more dataset(s) would improve the generalization and strengthen the models and the performances. Third, the anonymization of the features in the dataset was not something that could be changed, but it did prevent finding specific interpretations of the model's results. The only thing that could be said was which feature it was, but there was no indication as to what that feature contained information about relating to the APS system.

5.4 Future Work

Based on the results in this project, future work could be done to improve the model's performance, such as hyperparameter tuning. In this project, the parameter search was limited due to computational restraints, so expanding the search space may lead to better results, especially for the models Random Forest and XGBoost. Testing the models on additional

datasets or more recent data might also provide better results. This would help evaluate the models on datasets other than the ones used in the project. This would test the model on how well it would perform in a real-world situation instead of in a small, controlled environment. Finally, future work could be done to test the datasets for different models or additional feature engineering techniques to see if a different model not tested in this project is better suited for the classification task, or if changing parameters or features improved the performance of the six models in this project.

References

- GeeksforGeeks. (2020, September). *Evaluation Metrics in Machine Learning*. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>
- GeeksforGeeks. (2021, September 18). *XGBoost*. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/xgboost/>
- IBM. (2021, October 4). *What is the k-nearest neighbors (KNN) algorithm?* Ibm.com. <https://www.ibm.com/think/topics/knn>
- IBM. (2023, December 12). *Support Vector Machine*. IBM. <https://www.ibm.com/think/topics/support-vector-machine>
- Galli, S. (2023, March 13). *Dealing with Imbalanced Datasets in Machine Learning: Techniques and Best Practices*. Train in Data's Blog. <https://www.blog.trainindata.com/machine-learning-with-imbalanced-data/>
- GeeksforGeeks. (2017, May 9). *Logistic Regression in Machine Learning*. GeeksforGeeks. [Logistic Regression in Machine Learning - GeeksforGeeks](https://www.geeksforgeeks.org/logistic-regression-in-machine-learning/)
- GeeksforGeeks. (2024, February 22). *Random Forest Algorithm in Machine Learning*. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
- Heroshe Mihindukulasuriya. (2025). *Different Types of Decision Trees and Their Uses* | Creatly. [Different Types of Decision Trees and Their Uses | Creatly](https://www.creatly.com/decision-trees/)
- Hussain, S.A., Prasad V, P.N.S.B.S.V., Kodali, R. *et al*. Predicting and Categorizing Air Pressure System Failures in Scania Trucks using Machine Learning. *J. Electron. Mater.* **53**, 3603–3613 (2024). <https://doi.org/10.1007/s11664-024-11115-8>
- IDA2016Challenge [Dataset]. (2016). UCI Machine Learning Repository. <https://doi.org/10.24432/C5V60Q>.
- K. T. Selvi, N. Praveena, K. Pratheeksha, S. Ragunathan and R. Thamilselvan, "Air Pressure System Failure Prediction and Classification in Scania Trucks using Machine Learning," 2022 Second International Conference on Artificial Intelligence and Smart Energy (ICAIS), Coimbatore, India, 2022, pp. 220-227, doi: 10.1109/ICAIS53314.2022.9742716. keywords: {Industries;Machine learning algorithms;Costs;Prediction algorithms;Classification algorithms;Automobiles;Task analysis;Air Pressure Sytem;Convolution;Class Imbalance;Classification;Accuracy;Predictive Maintenance},
- Scikit-learn. (2019). *StandardScaler*. Scikit-learn.org. [StandardScaler — scikit-learn 1.8.0 documentation](https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html)
- Taghandiki, K., DallakehNejad, M., & Asiabarak, H. R. (2023). Minimizing the repair cost of the air pressure system of scania trucks using a deep learning algorithm.